Table des matières

Cours 08 — Mettre en place l'intégration continue 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Une cible pour la CI, et son entrepôt jetable · Fiabiliser et faire tourner 2
Étape 2 — Le linteur · Fiabiliser et faire tourner 4
Étape 3 — La fraîcheur de la caisse · STAGING 7
Étape 4 — Le contrat du fait des ventes · MARTS 8
Étape 5 — La CI, sur une copie neuve du dépôt · Fiabiliser et faire tourner 9
Étape 6 — La branche qui casse · Fiabiliser et faire tourner 11
À retenir 12

Cours 08 — Mettre en place l'intégration continue

Cours 8/9 · La Brûlerie — Formation Analytics Engineering

`STAGING`	`INTERMEDIATE`	`MARTS`
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
✓ ici · Fiabiliser et faire tourner, ce qui traverse les trois couches : les tests, l'incrémental, l'intégration continue	✓	✓

Ce que vous allez faire

Jusqu'ici, c'est vous qui lanciez les tests, quand vous y pensiez. L'intégration continue, la CI (*continuous integration*), les lance à chaque changement proposé, sur une copie neuve du dépôt et un entrepôt jetable, et refuse le changement qui casse. Ce cours en pose les pièces : une cible et un entrepôt pour la CI, le linteur qui vérifie la forme du SQL, la fraîcheur de la caisse, un contrat sur le fait des ventes, puis le script qui enchaîne le tout. Il finit par une branche qui renomme une colonne, et que la CI refuse.

la cible `ci` et son entrepôt jetable → le linteur → la fraîcheur de la caisse → le contrat de `fct_ventes` → la CI sur une copie neuve → la branche qui casse

© 2026 iAlexMG. Tous droits réservés.

Les fichiers à écrire se donnent en pseudocode, et le code complet est dans `Codes_08_AE.md`, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, dbt, le linteur, Git et la CI ; la seconde pour le notebook de DuckDB, qui s'ouvre à l'étape 1. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set DBT_PROJECT_DIR=02_ae
REM Attendu : rien ne s'affiche.
dbt build
REM Attendu : 43 of 43, dont 1 incremental model, puis Done. PASS=43 WARN=0 ERROR=0.
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
sqlfluff --version
REM Attendu : sqlfluff, version 4.3.0.
```

Ce cours part du projet tel que le cours 07 l'a laissé, commité. S'il reste un fichier modifié, reprenez la fin du cours 07 avant de continuer. sqlfluff, le linteur, s'est installé avec dbt, par le `requirements.txt` de la formation.

Étape 1 — Une cible pour la CI, et son entrepôt jetable · Fiabiliser et faire tourner

La CI vérifie le code, pas les données. En entreprise, elle tourne sur une machine neuve, qui n'a pas l'entrepôt de votre poste : elle bâtit donc le projet dans son propre entrepôt, qui n'a que la structure des sources, les tables `raw_…` de l'entrepôt et leurs colonnes, sans une ligne. Ici, `plateforme\entrepot\ci.duckdb` en tient lieu, à côté de votre entrepôt.

Elle a sa propre connexion, une cible `ci` dans un profil à elle. Ce profil vit dans le dépôt, dans un dossier `ci` : il ne porte qu'un nom de fichier, rien de secret, et il doit voyager avec le code. Votre `profiles.yml`, lui, reste hors du dépôt.

1.1 Créer le dossier, puis le profil, dans la première fenêtre.

```
mkdir 02_ae\ci
notepad 02_ae\ci\profiles.yml
```

Le profil porte le nom que `dbt_project.yml` lui donne, comme celui du cours 03, et le chemin de l'entrepôt jetable, `plateforme/entrepot/ci.duckdb`. Accepter la création, puis écrire :

```
# 02_ae/ci/profiles.yml : la connexion de la CI, à un entrepôt jetable ; versionnée, rien n'y est secret
____:
  target: ci
  outputs:
```


© 2026 iAlexMG. Tous droits réservés.

```
    ci:
      # 1. le moteur, et l'entrepôt jetable, vu depuis le dossier d'où l'on lance dbt
      type: duckdb
      path: ____
```

Enregistrer, puis fermer le Bloc-notes.

1.2 Tirer la structure des sources. DuckDB garde, pour chaque table, l'instruction qui la crée : la colonne `sql` de `duckdb_tables()`. `-noheader -list` ne garde que les valeurs, et `>` les écrit dans un fichier, que la CI rejouera.

```
duckdb -readonly plateforme\entrepot\brulerie.duckdb -noheader -list -c "select sql from duckdb_tables() where
starts_with(table_name, 'raw_') order by table_name" > 02_ae\ci\structure_sources.sql
REM Attendu : rien ne s'affiche, l'invite revient.
type 02_ae\ci\structure_sources.sql
REM Attendu : huit lignes, de CREATE TABLE raw_consentements(membre_id VARCHAR, … à CREATE TABLE
REM raw_tickets(ticket_id VARCHAR, …, filename VARCHAR) ; raw_courbes, à Data Engineering, en est.
```

Le fichier se refait par la même commande le jour où la `RAW` change de forme.

1.3 Monter l'entrepôt jetable.

```
duckdb plateforme\entrepot\ci.duckdb -f 02_ae\ci\structure_sources.sql
REM Attendu : rien ne s'affiche ; ci.duckdb paraît dans plateforme\entrepot, à côté de brulerie.duckdb.
```

1.4 Bâtir le projet sur la cible `ci`. `--target` choisit la cible, et `--profiles-dir` dit où trouver son profil.

```
dbt build --target ci --profiles-dir 02_ae\ci
REM Attendu : Unable to do partial parsing, puis Concurrency: 1 threads (target='ci'), 43 of 43
REM et Done. PASS=43 WARN=0 ERROR=0.
```

dbt prévient d'abord qu'il relit tout le projet : la cible a changé depuis sa dernière commande. Il le redira à chaque passage d'une cible à l'autre, sans effet.

Si la commande s'arrête sur	**La cause, et le geste**
`Catalog Error: Table with name "raw_consentements" already exists!`	`ci.duckdb` porte déjà la structure : l'effacer par `del plateforme\entrepot\ci.duckdb`, puis reprendre l'étape 1.3
`Could not find profile named 'brulerie'`	le profil de la CI ne porte pas le nom du projet : reprendre l'étape 1.1

1.5 Écarter ce que dbt a laissé à côté du profil. À sa première lecture d'un profil, dbt écrit dans le même dossier un `.user.yml`, un identifiant anonyme de votre poste. Celui de votre profil personnel vit dans `Projet_Brulerie`, hors du dépôt ; celui-ci est dans `02_ae\ci`, et il entrerait au prochain commit.

```
dir /b 02_ae\ci
REM Attendu : .user.yml, profiles.yml et structure_sources.sql.
notepad 02_ae\.gitignore
```


© 2026 iAlexMG. Tous droits réservés.

Ajouter ces deux lignes au bout du fichier :

```
# L'identifiant que dbt écrit à côté du profil de la CI : il est propre au poste
.user.yml
```

Enregistrer, puis fermer le Bloc-notes.

1.6 Regarder ce que la CI a bâti. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « AE Cours 08 ». Dans la première cellule, attacher les deux entrepôts :

```
attach if not exists 'plateforme/entrepot/brulerie.duckdb' as brulerie (read_only);
attach if not exists 'plateforme/entrepot/ci.duckdb' as ci (read_only);
-- Attendu : brulerie et ci paraissent sous Attached databases.
```

Puis, dans une cellule neuve, compter le fait des ventes des deux côtés. Chaque sous-requête rend une seule valeur, comme celle du filtre au cours 07.

```
-- 1. le fait des ventes sur votre entrepôt, puis sur celui de la CI, et les jours de sa dimension
select (select count(*) from brulerie.fct_ventes) as ventes_poste,
       (select count(*) from ci.fct_ventes) as ventes_ci,
       (select count(*) from ci.dim_date) as jours_ci;
-- Attendu : 2509, 0 et 792.
```

Les 29 tests sont passés sur des tables vides : un test cherche des lignes en faute, et il n'y en a aucune. `dim_date`, fabriquée sans source, est la seule table de la CI qui ait des lignes. La CI prouve que le code se compile, tourne et garde sa forme ; elle ne dit pas que les chiffres sont justes. Ça, c'est le travail des tests sur votre entrepôt, et du témoin.

1.7 Fermer le notebook : le linteur et dbt vont ouvrir `ci.duckdb`. Fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient. Le notebook ne resservira plus dans ce cours.

Étape 2 — Le linteur · Fiabiliser et faire tourner

Un linteur lit le code sans l'exécuter, et signale ce qui s'écarte des règles de forme de l'équipe : le retrait, la casse, la longueur des lignes, les espaces. sqlfluff le fait pour le SQL. Son templater dbt compile d'abord chaque modèle, `ref()`, `source()` et `is_incremental()` compris, avec le profil de la CI : il lit le SQL que dbt enverrait. Chaque règle a un code, `LT` pour la mise en page, `CP` pour la casse, `ST` pour la structure.

Il lit sa configuration dans `.sqlfluff`, à la racine du dépôt. Les règles par défaut ne sont pas toutes celles du projet : une équipe en écarte, et écrit pourquoi, pour que personne ne les rallume sans le savoir.

© 2026 iAlexMG. Tous droits réservés.

La section	Ce qu'elle dit
`[sqlfluff]`	le dialecte, DuckDB ; le templater, dbt ; la longueur d'une ligne ; les règles écartées
`[sqlfluff:templater:dbt]`	où trouver le projet et le profil de la CI, depuis ce fichier, et la cible
`[sqlfluff:indentation]`	le contenu d'un bloc de Jinja, comme celui du snapshot, garde le retrait du SQL qui l'entoure
`[sqlfluff:rules:capitalisation.keywords]`	les mots-clés en minuscules
`[sqlfluff:rules:layout.long_lines]`	une ligne de commentaire peut dépasser

2.1 Écrire la configuration.

```
notepad 02_ae\.sqlfluff
```

Le fichier est un `.ini` : des sections entre crochets, des paires `clé = valeur`, et des commentaires après `#`. Accepter la création, puis écrire :

```
# 02_ae/.sqlfluff : la forme du SQL du projet, que le linteur fait respecter
[sqlfluff]
# 1. le moteur, et le templater qui compile le Jinja de dbt avant de lire
dialect = ____
templater = ____
# 2. la longueur d'une ligne de code, puis les règles écartées, avec leur raison :
# ST06 et ST09 réordonneraient les colonnes et les jointures sans rien corriger ;
# LT08 et LT09 déplieraient les tests courts au-delà de 20 lignes
max_line_length = 100
exclude_rules = ST06, ST09, LT08, LT09

[sqlfluff:templater:dbt]
# 3. le projet, et le profil de la CI, vus depuis ce fichier
project_dir = .
profiles_dir = ____
target = ____

[sqlfluff:indentation]
# 4. le contenu d'un bloc de Jinja garde le retrait du SQL qui l'entoure
template_blocks_indent = false

[sqlfluff:rules:capitalisation.keywords]
# 5. les mots-clés en minuscules, comme tout le SQL du projet
```


© 2026 iAlexMG. Tous droits réservés.

```
capitalisation_policy = ____

[sqlfluff:rules:layout.long_lines]
# 6. un commentaire peut dépasser : la première ligne de chaque fichier dit ce qu'il fait
ignore_comment_lines = true
```

Enregistrer, puis fermer le Bloc-notes.

2.2 Passer le projet au linteur. sqlfluff ne prend son templater que d'un `.sqlfluff` posé dans le dossier d'où on le lance ; la fenêtre reste dans `Projet_Brulerie`, et `--config` lui désigne celui du dépôt. Les trois dossiers nommés sont ceux qui portent du SQL : `target`, que dbt refait, reste dehors.

```
sqlfluff lint 02_ae\models 02_ae\tests 02_ae\snapshots --config 02_ae\.sqlfluff
REM Attendu : [dbt templater] Project Compiled., puis trois fichiers en FAIL : dim_date.sql, LT01
REM à la ligne 14 ; fct_ventes.sql, RF02 à la ligne 19 ; snp_prix.sql, LT02 à la ligne 13.
REM Puis All Finished!
```

Trois écarts, chacun avec sa ligne et sa règle. `LT01` veut une espace entre le nom que `generate_series` donne à sa table et la liste de ses colonnes, `jours (jour)`. `LT02` veut qu'une condition de `where` sur plusieurs lignes commence sous le `where`, chacune sur sa ligne, au même retrait. `RF02` vise le filtre du fait des ventes : l'étape 2.4 s'en occupe. Si vos fichiers s'écartent des codes des cours, le linteur en trouve davantage : ce sont les vôtres, et le geste est le même.

Si le linteur s'arrête sur	La cause, et le geste
`Attempt to set templater to dbt failed. Using jinja templater.`	`--config 02_ae\.sqlfluff` manque à la commande : la reprendre telle quelle
`Error: No such option '--profiles-dir'.`	l'option est de dbt, pas de sqlfluff : le profil se déclare dans `.sqlfluff`, sous `[sqlfluff:templater:dbt]`
`IO Error: Cannot open file "…\ci.duckdb"`	le notebook tient l'entrepôt de la CI : quitter DuckDB dans la seconde fenêtre

2.3 Laisser le linteur corriger, puis relire ce qu'il a fait. `fix` réécrit les fichiers ; on ne le croit qu'après avoir lu la différence.

```
sqlfluff fix 02_ae\models 02_ae\tests 02_ae\snapshots --config 02_ae\.sqlfluff
REM Attendu : dim_date.sql et snp_prix.sql FIXED, puis 2 fixable linting violations found et
REM [1 unfixable linting violations found].
git -C 02_ae diff
REM Attendu : as jours(jour) devient as jours (jour) ; dans snp_prix.sql, champ in ('CREATION',
REM 'PRIX') passe sous le where ; le .gitignore gagne ses deux lignes.
```

Git prévient aussi que les fins de ligne de ces deux fichiers passeront en CRLF : le linteur écrit celles d'Unix, et Git les ramène à celles de Windows, sans rien changer au contenu.

© 2026 iAlexMG. Tous droits réservés.

2.4 Qualifier la colonne du filtre, à la main. Le linteur lit le SQL que dbt enverrait : le fait existe dans `ci.duckdb` depuis l'étape 1.4, donc dbt compile la branche incrémentale, et son filtre. Dans `where exporte_le > (select max(exporte_le) from {{ this }})`, deux tables sont en jeu, `int_ventes` et le fait lui-même, et rien ne dit de laquelle vient chaque `exporte_le`. Ce n'est pas qu'une affaire de style : une colonne que la sous-requête n'a pas se cherche dans la requête qui l'entoure. Si la table du fait n'avait pas encore cette colonne, après une passe lancée sans `--full-refresh` à l'étape 4 du cours 07, la sous-requête prendrait celle d'`int_ventes`, et DuckDB s'arrêterait sur `WHERE clause cannot contain aggregates!`, qui ne nomme ni la colonne ni la table. Un alias sur `{{ this }}`, et la colonne devant lui, lèvent le doute.

```
notepad 02_ae\models\marts\fct_ventes.sql
```

Seule la ligne du filtre change :

```
where exporte_le > (select max(____.exporte_le) from {{ this }} as ____)
```

Enregistrer, puis fermer le Bloc-notes. `fix` ne fait pas ce geste : décider de quelle table vient une colonne demande de connaître la requête.

2.5 Relancer le linteur.

```
sqlfluff lint 02_ae\models 02_ae\tests 02_ae\snapshots --config 02_ae\.sqlfluff
REM Attendu : [dbt templater] Project Compiled., puis All Finished! sans aucun FAIL.
```

Étape 3 — La fraîcheur de la caisse · STAGING

Un test dit si les données sont justes ; la fraîcheur dit si elles sont récentes. `dbt source freshness` lit, pour chaque source qui la déclare, l'instant de son dernier chargement, le compare à l'heure présente, et passé un seuil, avertit ou rend une erreur. Les seuils suivent la cadence de la source : la caisse exporte chaque soir, donc un jour sans export avertit, et deux font une erreur.

La `RAW` n'a pas de colonne de chargement. L'instant de l'export, que le cours 07 tire du nom du fichier, en tient lieu : `loaded_at_field` accepte une expression SQL. Les autres tables de la `RAW` n'ont pas d'instant d'export, et leur fraîcheur ne se mesure pas ici.

3.1 Déclarer la fraîcheur des deux tables de la caisse, dans la première fenêtre.

```
notepad 02_ae\models\staging\_sources.yml
```

Sous `raw_tickets`, après sa description, ajouter le commentaire 3 et le bloc `config:`. Les deux seuils comptent des jours :

```
      - name: raw_tickets
        description: "Les tickets de la caisse, un par ligne et par export."
        # 3. la caisse exporte chaque soir : sa fraîcheur se lit à l'instant de son dernier export
        config:
          loaded_at_field: "strptime(replace(split_part(filename, '_', 3), '.csv', ''), '%Y%m%d%H%M')"
          freshness:
            warn_after: {count: ____, period: day}
            error_after: {count: ____, period: day}
```


© 2026 iAlexMG. Tous droits réservés.

Puis le même bloc `config:`, sans le commentaire, sous la description de `raw_lignes`. Enregistrer, puis fermer le Bloc-notes.

3.2 Mesurer la fraîcheur, puis lire le code que la commande rend. `%ERRORLEVEL%` garde le code de sortie de la dernière commande : 0 si tout va bien.

```
dbt source freshness
REM Attendu : 2 of 2 ERROR STALE freshness of brulerie.raw_lignes et de brulerie.raw_tickets, puis Done.
echo %ERRORLEVEL%
REM Attendu : 1.
```

La commande a raison : le mois est figé, et le dernier export date du 30 juin 2025 à 18 h 07. En exploitation, elle tourne chaque nuit, juste avant `dbt build`, et son code 1 arrête la chaîne : on ne bâtit pas un rapport sur une caisse muette. Un avertissement, lui, rend 0 : la chaîne continue, et il ne sert que si quelqu'un lit le journal. La fraîcheur ne va pas dans la CI : elle regarde les données, et la CI n'en a pas.

Étape 4 — Le contrat du fait des ventes · MARTS

Les rapports lisent `fct_ventes` par le nom et le type de ses colonnes : c'est son interface. Un contrat la fige dans le YAML, colonne par colonne, avec son type. Avant de bâtir le modèle, dbt vérifie que sa requête rend exactement ces colonnes, sous ces types ; sinon, il refuse de le bâtir. Un test regarde les lignes, et ne voit rien sur une table vide ; le contrat regarde la forme, et la voit sans données. C'est ce qui le rend précieux à la CI.

Sur un fait incrémental, dbt veut aussi savoir que faire si les colonnes changent d'une passe à l'autre : `on_schema_change: fail` arrête la passe, plutôt que de laisser la table et la requête diverger.

La colonne	Son type	D'où il vient
`cle_date`	`date`	la date de la vente
`cle_produit`, `cle_membre`	`varchar`	`md5` rend du texte
`ticket_id`, `ligne`, `quantite`	`integer`	typés au `STAGING`
`exporte_le`	`timestamp`	`strptime` rend un horodatage
`montant_brut`	`decimal(18, 2)`	un entier fois un `decimal(10, 2)`
`rabais`, `montant`	`decimal(10, 2)`	typés au `STAGING`

4.1 Écrire le contrat, dans la première fenêtre.

```
notepad 02_ae\models\marts\_marts.yml
```

Sous `fct_ventes`, le commentaire 3 change, le bloc `config:` gagne deux réglages, et chaque colonne prend son type, dans l'ordre où le modèle les rend. Un contrat veut toutes les colonnes, pas seulement celles que les tests surveillent :

© 2026 iAlexMG. Tous droits réservés.

```yaml
  # 3. le fait, bâti par passes et tenu par son contrat : chaque colonne typée, chaque clé dans sa dimension
  - name: fct_ventes
    description: "Le fait des ventes, une ligne par ligne d'un ticket payé."
    config:
      materialized: incremental
      incremental_strategy: delete+insert
      unique_key: [ticket_id, ligne]
      on_schema_change: ____
      contract:
        enforced: ____
    columns:
      - name: cle_date
        data_type: date
        # puis, à écrire : les tests de cle_date, inchangés ; de même sous cle_produit et cle_membre, chacune
        # avec son data_type au-dessus de ses tests
      # puis, à écrire : ticket_id, ligne, exporte_le, quantite, montant_brut, rabais et montant, chacune avec
      # son nom et son data_type, dans cet ordre
```

Enregistrer, puis fermer le Bloc-notes.

4.2 Tout lancer.

```
dbt build
REM Attendu : 43 of 43, puis Done. PASS=43 WARN=0 ERROR=0.
```

Le contrat n'ajoute aucun nœud : il se vérifie au moment de bâtir le modèle, et le modèle le passe.

Si dbt s'arrête sur	**La cause, et le geste**
`Invalid value for on_schema_change: ignore`	le réglage manque : un fait incrémental sous contrat veut `on_schema_change: fail`
`This model has an enforced contract that failed.`	une colonne manque, est de trop ou n'a pas le bon type : le tableau du message dit laquelle

Étape 5 — La CI, sur une copie neuve du dépôt · Fiabiliser et faire tourner

Les pièces sont posées ; il reste à les enchaîner, toujours dans le même ordre, sans rien qui vienne de votre poste. En entreprise, une plateforme (GitHub Actions, GitLab CI) lance ces étapes à chaque branche proposée, sur une machine neuve. Ici, un court script Python joue ce rôle. Il crée un dossier temporaire, y clone le dépôt, y monte l'entrepôt jetable, lance le linteur, puis `dbt build`, et s'arrête à la première étape en échec, avec un code d'erreur.

© 2026 iAlexMG. Tous droits réservés.

La copie neuve est le point : la CI ne voit que ce qui est commité. Un fichier oublié, un réglage qui ne vit que sur votre poste, votre profil personnel, rien de cela n'y est, et la CI tombe là où votre poste passerait. Le dossier temporaire tient la copie sous `02_ae` et l'entrepôt à côté, comme `Projet_Brulerie` : les commandes sont celles des étapes 1 à 4.

Pour remplir	**L'outil**
le dossier du dépôt, où que le script soit lancé	`Path(__file__).resolve().parents[1]` : le fichier, puis deux crans plus haut, `ci` puis `02_ae`
un dossier qui s'efface tout seul	`with tempfile.TemporaryDirectory() as dossier:`
le dossier de l'entrepôt jetable, que DuckDB ne crée pas	`(Path(dossier) / "plateforme" / "entrepot").mkdir(parents=True)`, la première ligne sous le `with`
lancer une commande dans ce dossier	`subprocess.run(etape, cwd=dossier, shell=True)`, dont le `returncode` vaut 0 si tout va bien
arrêter le script avec un code d'erreur	`sys.exit("message")`, qui affiche le message et rend 1

5.1 Écrire le script.

```
notepad 02_ae\ci\verifier.py
```

Dans une chaîne Python, les chemins s'écrivent avec des barres obliques, `02_ae/ci/…` : les outils de Windows les comprennent, et une barre inverse y commencerait une séquence d'échappement. Accepter la création, puis écrire :

```python
# 02_ae/ci/verifier.py : la CI locale, qui vérifie le dernier commit sur une copie neuve du dépôt
import os
import subprocess
import sys
import tempfile
from pathlib import Path

# 1. le dépôt, où que le script soit lancé ; dbt cherchera le projet dans 02_ae, comme chez vous
DEPOT = Path(__file__).resolve().parents[____]
os.environ["DBT_PROJECT_DIR"] = "02_ae"
# 2. les étapes, dans l'ordre : la copie, l'entrepôt jetable, la forme, puis le fond
ETAPES = [
    f'git clone --quiet "{DEPOT}" 02_ae',
    # puis, à écrire : les commandes des étapes 1.3, 2.2 et 1.4, dans cet ordre, les chemins en barres obliques
]
# 3. un dossier neuf, qui s'efface à la fin, avec le dossier de l'entrepôt jetable
with tempfile.____() as dossier:
```


© 2026 iAlexMG. Tous droits réservés.

```
    ____
    for etape in ETAPES:
        print(f"\n> {etape}", flush=True)
        # 4. la CI s'arrête à la première étape en échec, avec un code d'erreur
        if subprocess.run(etape, cwd=____, shell=True).returncode != 0:
            sys.exit(f"\nCI en échec : {etape}")
print("\nCI au vert : le dernier commit passe la forme et le fond, sur une copie neuve.")
```

Enregistrer, puis fermer le Bloc-notes.

5.2 Versionner le cours : la CI ne verra que ce qui est commité.

```
git -C 02_ae add .gitignore .sqlfluff ci models snapshots
git -C 02_ae commit -m "Cours 08 : le linteur, la fraîcheur, le contrat et la CI"
REM Attendu : 10 files changed.
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```

Les dix : le profil de la CI, la structure des sources et le script ; `.gitignore` et `.sqlfluff` ; les deux fichiers que le linteur a corrigés, et le fait des ventes qualifié à la main ; `_sources.yml` et `_marts.yml`. Le `.user.yml` de `02_ae\ci` reste dehors.

5.3 Lancer la CI. Comptez une vingtaine de secondes : le linteur compile le projet, puis dbt le bâtit.

```
python 02_ae\ci\verifier.py
REM Attendu : quatre étapes, chacune annoncée par >, dont All Finished! et Done. PASS=43, puis
REM CI au vert : le dernier commit passe la forme et le fond, sur une copie neuve.
```

Le dossier temporaire s'est effacé : rien n'a changé dans `Projet_Brulerie`.

Étape 6 — La branche qui casse · Fiabiliser et faire tourner

En équipe, un changement se propose sur une branche, une ligne de commits à côté de la branche principale, et la CI le vérifie avant qu'il la rejoigne. Un collègue trouve `montant_brut` trop long, et propose `brut`. Le changement tient en un mot, le projet se bâtit, et ses tests passent, même sur vos vraies données : aucun ne lit cette colonne. Les rapports qui la lisent, eux, tomberaient le lendemain.

6.1 Créer la branche, dans la première fenêtre. `switch -c` crée une branche et s'y place.

```
git -C 02_ae switch -c brut-plus-court
REM Attendu : Switched to a new branch 'brut-plus-court'.
notepad 02_ae\models\marts\fct_ventes.sql
```

Remplacer `as montant_brut` par `as brut`. Enregistrer, puis fermer le Bloc-notes.

6.2 Commiter le changement, puis lancer la CI. `commit -am` prend tous les fichiers suivis qui ont changé.

© 2026 iAlexMG. Tous droits réservés.

```
git -C 02_ae commit -am "fct_ventes : montant_brut devient brut"
REM Attendu : 1 file changed, 1 insertion(+), 1 deletion(-).
python 02_ae\ci\verifier.py
REM Attendu : All Finished! au linteur, puis ERROR creating sql incremental model main.fct_ventes,
REM le tableau du contrat, Done. PASS=35 WARN=0 ERROR=1 SKIP=7, et CI en échec : dbt build …
```

Le contrat a refusé de bâtir le fait, et dbt a sauté ses sept tests. Son message dit pourquoi :

```
  This model has an enforced contract that failed.
  | column_name  | definition_type | contract_type | mismatch_reason       |
  | ------------ | --------------- | ------------- | --------------------- |
  | brut         | DECIMAL(18,2)   |               | missing in contract   |
  | montant_brut |                 | DECIMAL(18,2) | missing in definition |
```

`brut` sort du modèle sans figurer au contrat ; `montant_brut` est au contrat, et le modèle ne le rend plus. Sur un entrepôt vide, le contrat était le seul à pouvoir le voir. Si le collègue tient au nouveau nom, il le change aussi dans le contrat, dans la même branche : la relecture voit alors l'interface changer, et prévient ceux qui la lisent.

6.3 Abandonner la branche. `switch -` revient à la branche d'avant, et `branch -D` efface une branche qui n'a jamais rejoint la principale.

```
git -C 02_ae switch -
REM Attendu : Switched to branch 'main', ou 'master' selon votre Git.
git -C 02_ae branch -D brut-plus-court
REM Attendu : Deleted branch brut-plus-court.
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```

`fct_ventes.sql` a retrouvé `montant_brut` : le fichier suit la branche où l'on se trouve.

À retenir

- La CI rejoue les mêmes étapes à chaque changement, sur une copie neuve du dépôt et un entrepôt qui n'a que la structure des sources : la forme par le linteur, puis le fond par `dbt build`. Elle ne voit que ce qui est commité.
- Sur des tables vides, un test ne prouve rien ; le contrat fige le nom et le type de chaque colonne du fait, et refuse le changement qui les touche sans le dire.
- La fraîcheur regarde les données, pas le code : elle tourne avant le `dbt build` de la nuit, et son erreur arrête la chaîne.

© 2026 iAlexMG. Tous droits réservés.